TUTORIAL 7

As mentioned, TAs will resolve midsem doubts for the first 25 minutes

This week, the TAs will help students perform basic matrix operations such as printing matrix, the addition of matrices, multiplication of matrices, and finally matrix transpose.

Matrix (? → TAs will explain what is matrix, the dimension, and finally how it can be interpreted as a list of lists).

For this tutorial, two square matrices would be considered, “Matrix_A” and “Matrix_B”.
Square Matrix → Rows and Columns have the same dimension (i.e, their number is the same)

```
Matrix_A = [
      [1, 2, 3],
      [4, 5, 6],
      [7, 8, 9]
      ]

Matrix_B = [
      [2, 3, 1],
      [4, 5, 4],
      [1, 3, 4]
      ]
```

Let’s print the first matrix (TAs will mention,how each list can be considered as individual row)

```
print(Matrix_A)
```

And the output is:

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Okay, cool! But, can it be mode more illustrative? Something relatable?
Let’s create a generic function that accepts such list of lists and prints in the matrix form

```
def printMatrix(M):
      n_row = len(M)
      ## Number of rows

      for r in range(n_row):
            print(*M[r])
            ##will print space separated
            ## rows of a matrix

##Let's call the function
printMatrix(Matrix_A)
```

And the output is:

```
1 2 3
4 5 6
7 8 9
```

Looks good!!

Similarly, Matrix_B can be printed. The TAs would ask the students to print the same.

Now, The TAs would mention 4 operations, out of which Matrix Multiplication should be given as exercise in the classroom.

A. **Matrix Addition**
B. **Matrix Subtraction**
C. **Matrix Transpose**
D. **Matrix Multiplication**(classroom exercise→Not graded)

Matrix Addition

```
def addMatrices(M1, M2):
     ## for square matrices only

     n_row = len(M1) ##or len(M2)
     n_col = len(M1[0]) ##or len(M1[0])

     resultant = [
          [ M1[i][j]+ M2[i][j] for j in range(n_col)]
          for i in range(n_row)

          ]

     ##calling the previously defined printMatrix()
     printMatrix(resultant)

##calling add matrices

addMatrices(Matrix_A, Matrix_B)
```

Let's see the output:

```
3 5 4
8 10 10
8 11 13
```

It has worked.
Similarly Matrix Subtraction could be shown, and students should be asked to create a function for that.

MATRIX TRANSPOSE:

```
def transpose_matrix(M):
      n_row = len(M)
      n_col = len(M[0])

      transpose = [
            [M[j][i] for j in range(n_row)]
            for i in range(n_col)

            ]

      ##Again our friend printMatrix comes to rescue

      printMatrix(transpose)

## Let's call transpose_matrix for matrix_B

print("Matrix B:")
printMatrix(Matrix_B)
print("After Transposing")
transpose_matrix(Matrix_B)
```

Let's see the output:

```
Matrix B:
2 3 1
4 5 4
1 3 4
After Transposing
2 4 1
3 5 3
1 4 4
```

With this, the concepts of **Function, Function Call within Function, List comprehension for multiple dimensions, Matrix manipulation** would have been covered.

The student should be made realized how creating a simple **printMatrix()** function has eased our task to such a great extent. This will explain the **reusability** to them.

Finally, the students would be asked to perform matrix multiplication by themselves and could seek the TAs for verification as well.

This tutorial on matrix operations is necessary for solving many of the programming problems and to some extent helps in developing intuition with some of the assignment questions.